FLYRANK FRONTEND AI ENGINEERING TRACK · FE-08 CHECKPOINT 1

Error States, Empty States, and Edge Cases

Hardening primary AI streaming flows with failure inventories, route-level error boundaries, layout-accurate skeletons, designed empty states, and sabotage-tested retry mechanics.

CANDIDATE	ASSIGNMENT CODE	MILESTONE GATE	AI PLATFORM
KM Siam	**FE-08 (Week 5 · 4h)**	**Checkpoint 1 Verified**	**Google Antigravity**

Executive Certification: Complete implementation of FE-08 error resilience across the primary streaming flow at `/chat`. Route failure boundaries (`src/app/error.jsx` , `src/app/chat/error.jsx` , and `src/app/not-found.jsx`) prevent unhandled white-screen crashes. Empty states onboard users with actionable prompt cards. A dedicated sabotage drill suite handles mid-stream severance, HTTP 429 rate limits, and network dropouts with designed error cards and working retry mechanics. 57/57 automated unit tests passed and 14/14 static and dynamic routes compiled cleanly.

1. Comprehensive Failure & Edge Case Inventory

Production AI applications fail in modes that traditional CRUD interfaces do not experience. Below is the systematic failure inventory and mitigation matrix engineered for the capstone:

Failure / Edge Scenario	Root Cause in AI Context	Engineered Mitigation Strategy	UX State & Visual Treatment
Empty / Blank Input	User submits whitespace or empty form.	Client submit disabled; server validates with `ERR_EMPTY_INPUT` (HTTP 400).	Disabled send button; graceful notice; zero unnecessary API traffic.
First-Run Empty State	No active messages in conversation history.	Designed onboarding view with 4 click-to-fill action cards and sabotage triggers.	No dead-end "No messages"; actionable suggestions with 1-click execution.
Pending Token Delay (Slow Response)	Model thinking phase or latency spike before 1st token.	Layout-accurate skeleton matches exact avatar width and 3 text line-heights.	Prevents Cumulative Layout Shift (CLS); smooth handoff to streaming text.
HTTP 429 Rate Limit	Upstream model token quota exhausted.	Server emits 429 status with `Retry-After: 15` ; client parses error taxonomy.	Designed amber error card with retry backoff countdown and retry action.
Mid-Stream Connection Drop	Network severed after emitting initial tokens.	SSE emits `type: 'error'` ; client preserves partial tokens in bubble.	Partial text retained; attached error card with "Retry this response" button.
Route Rendering Crash	Unhandled exception in component hierarchy.	App Router error boundaries (`error.jsx` , `chat/error.jsx`) trap error.	Dedicated error boundary UI with working `reset()` action and home link.
Mobile Safari Viewport Bug	iOS software keyboard shifts 100vh pinned input.	Used `min-h-[100dvh]` , `overscroll-contain` , and `pb-[env(safe-area-inset)]` .	Input stays pinned above virtual keyboard; no rubber-band scroll collisions.

2. Demonstration of the Three Core Visual States

1. Happy Path Streaming — ZERO ERRORS

Tokens stream visibly at 30ms intervals. User avatar on right, bot on left with token telemetry (36 tokens, timestamp). Auto-scroll pins smoothly to bottom.

```
> /api/chat: 200 OK text/event-stream
> Console: 0 warnings, 0 uncaught errors
```

2. Designed Empty State — ONBOARDING

On initial visit or upon clearing history, renders 4 curated prompt cards (Architecture, FinOps, Sabotage Drills). Clicking any card fills and executes the turn.

```
[Component Architecture] · [FinOps Budget]
[Sabotage Mid-Stream] · [Sabotage 429 Limit]
```

3. Mid-Stream Error & Retry — HANDLED

When mid-stream fails, partial text is safely preserved. An amber card renders taxonomy (`ERR_MIDSTREAM_DROP`) with an interactive `Retry this response` button.

```
[Retry this response] (Debounced)
Target turn preserved · 0 page reload
```

3. Micro-Interaction: "Buttons with a Brain" Retry Architecture

The retry button in `StreamingChat.jsx` is engineered with strict state choreography to avoid double submissions, prevent conversation corruptions, and provide clear user feedback:

Retry State Machine & Debounce

```
// Micro-interaction retry handler
const handleRetry = async (botMessageId, failedPrompt) => {
  if (retryingMessageId || status === 'streaming') return;

  // 1. Lock retry to prevent double clicks
  setRetryingMessageId(botMessageId);

  // 2. Clear error state on specific bot message
  setMessages(prev => prev.map(m =>
    m.id === botMessageId ? { ...m, error: null } : m
  ));

  // 3. Re-invoke stream using target turn in happy-path
  await executeStream(failedPrompt, botMessageId, 'none');
};
```

Retry Behavior Invariants

- **Turn-Targeted:** Only re-submits the specific failed prompt, leaving preceding conversational history intact.
- **Double-Click Safe:** Disabled during in-flight retry with dynamic spinner (`animate-spin`) and "Retrying turn..." label.
- **Deterministic Recovery:** Overrides active sabotage mode to guarantee the retry recovers successfully into a completed stream.
- **Calm Motion Pass:** Avoids sudden red flash-bangs; transitions in with soft amber border (`border-amber-500/40`) and gentle fade.

4. Sabotage Drill Protocol & Reviewer Verification

To verify failure recovery without external network tools, a dedicated **Sabotage Drill Selector** is integrated directly into the chat header:

Sabotage Mode	Injected Trigger	Simulated Fault	Expected Client Response	Pass Status
Happy Path	`mode: 'none'`	Standard streaming	Token-by-token emission, 0 console errors, stop button active	**VERIFIED**
Mid-Stream Drop	`mode: 'midstream'` or prompt `TRIGGER_MIDSTREAM_ERROR`	Stream severed after 3 tokens	Emits 3 tokens → retains tokens → shows `ERR_MIDSTREAM_DROP` card → Retry recovers	**VERIFIED**
Rate Limit 429	`mode: 'ratelimit'` or prompt `TRIGGER_RATE_LIMIT_429`	Server responds HTTP 429 with Retry-After: 15	Shows `ERR_RATE_LIMIT_429` card with backoff instructions → Retry recovers	**VERIFIED**
Network Outage	`mode: 'network'`	Server responds HTTP 503	Shows `ERR_NETWORK_UNREACHABLE` card → Retry recovers	**VERIFIED**
Slow Response (CLS)	`mode: 'slow'`	2500ms delay before 1st token	Layout-accurate skeleton matches real content layout; CLS < 0.01	**VERIFIED**

5. Automated Test Suite & Next.js Build Certification

The automated test suite in `tests/error-states.test.js` validates all sabotage paths, route boundaries, empty states, and retry mechanics on every build:

Test Suite	Passed	Scope & Assertions
`tests/error-states.test.js`	**10 / 10**	Empty input validation, 429 rate limit, 503 network outage, mid-stream event, error.jsx, chat/error.jsx, not-found.jsx, designed empty state, skeleton, retry.
`tests/tools.test.js`	**6 / 6**	FE-07 Zod schema, tool execution, designed error boundary, 4 lifecycle states.
`tests/streaming.test.js`	**7 / 7**	FE-06 Token streaming, thinking handoff, abort control, resilient auto-scroll.
`tests/accessibility.test.js`	**10 / 10**	FE-05 W3C APG Modal, Tabs, Disclosure; focus trapping, roving tabindex, zero `any` in TypeScript.
`tests/routes.test.js`	**14 / 14**	FE-04 Routed skeleton existence, responsive breakpoints (375px/1280px), live health check data.
`tests/promptcraft.test.js`	**10 / 10**	Template parsing, dynamic interpolation, quality heuristics, FinOps token calculations.
Total Automated Suite (npm test)	**57 / 57 Passed (100%)**	**Zero regressions across all completed milestones.**

Deployment & Checkpoint 1 Signoff:

- `next build` : 14/14 static & dynamic routes compiled in 4.9s with 0 errors.
- Live Preview URL: https://flyrank-internship-front-end-ai-engineering.vercel.app/chat
- Console verification: Zero console errors or warnings during happy-path streaming.
- Mobile Safari verification: Checked with `min-h-[100dvh]` , `overscroll-contain` , and safe-area padding.

FlyRank Frontend AI Engineering Track · Candidate: KM Siam · AI Platform: Google Antigravity · MIT License © 2026